

Coding Best Practices Demonstration

Two versions of a command-line fortune teller

Deb Samanta · GitHub: dsamanta12344

<https://github.com/dsamanta12344/coding-best-practices-activity>

This assignment was completed independently. The prompt asked for a partner; none was available. Both program versions and this write-up are submitted by Deb Samanta. The same files can be turned in by a partner if one is assigned later. GitHub was used for version control (feature branches, commits, and merged pull requests) to complete the bonus.

1. Program description

The program is a small command-line **Fortune Booth**. It greets a visitor, asks for a name, a birth month, and a lucky number, then prints three short readings: love, career, and luck. Both Python files implement the same visitor-facing behavior. The point of the activity is not the fortunes themselves. It is the difference in how the two files are structured, named, and documented.

Each version uses well more than three functions. The issues file splits work into `get_name`, `love_fortune`, `job_fortune`, `luck_fortune`, and `do_everything`. The fixed file uses focused helpers for input, parsing, fortune selection, and display.

Run either program with `python fortune_teller_issues.py` or `python fortune_teller_fixed.py`. The issues version asks for the month and lucky number three times. The fixed version asks once and reuses the answers. That difference is the DRY violation made visible at runtime.

2. Principles chosen

I selected three principles from the lecture list and wrote one file that violates all three on purpose, then a second file that follows all three.

Principle	What it means in this lab
DRY (Don't Repeat Yourself)	Knowledge that can change (month parsing, fortune picking, banners) lives in one place, not copy-pasted into three functions.
Single responsibility	A function has one reason to change: collect input, or choose a line, or print a reading — not all three at once.
Document your code	Comments and docstrings explain intent. They do not narrate syntax, lie about return values, or leave dead code in the file.

I did not pick KISS, YAGNI, or composition-over-inheritance as the graded three, but the issues file also shows extras that usually travel with bad DRY/SRP: unused globals, an unused import, and a leftover `database_url` for a feature nobody asked for.

3. DRY — Don't Repeat Yourself

How the issues file violates DRY

`love_fortune`, `job_fortune`, and `luck_fortune` are the same function with the fortune strings swapped. Each one re-prompts for birth month, re-implements the 12-branch month parser, re-parses the lucky number, rebuilds a list, and indexes it with $(m + \text{num}) \% 4$. If the month parser is wrong, it is wrong in three places. Adding a fourth category (for example health) would mean a fourth paste.

```
def love_fortune():
    month_text = input("Birth month (1-12 or name): ")
    month_text = month_text.strip().lower()
    if month_text == "1" or month_text == "january":
        m = 1
    elif month_text == "2" or month_text == "february":
        m = 2
    # ... ten more branches, then the same block again in
    # job_fortune() and luck_fortune()
```

Listing 1. Month parsing copy-pasted inside each fortune function.

How the fixed file follows DRY

Month names live in one dictionary. Parsing lives in `parse_month`. Choosing a line lives in `choose_line`. Visitor answers are collected once in `collect_visitor` and passed into `build_reading`. The three categories become data (three tuples) plus one picker, not three cloned functions.

```
def parse_month(text: str) -> int | None:
    cleaned = text.strip().lower()
    if cleaned in MONTH_NAMES:
        return MONTH_NAMES[cleaned]
    if cleaned.isdigit():
        value = int(cleaned)
        if 1 <= value <= 12:
            return value
    return None

def choose_line(lines: tuple[str, ...], seed: int) -> str:
    return lines[seed % len(lines)]

def build_reading(month: int, lucky_number: int) -> dict[str, str]:
    seed = month + lucky_number
    return {
        "love": choose_line(LOVE_LINES, seed),
        "career": choose_line(CAREER_LINES, seed),
        "luck": choose_line(LUCK_LINES, seed),
    }
```

Listing 2. One parser, one picker, three data sets. A new category is one tuple plus one dictionary entry.

4. Single responsibility

How the issues file violates SRP

`get_name` does not only get a name. It also prints the welcome banner and writes a global. `love_fortune` asks for month and lucky number, parses both, selects a line, and prints the section header plus the fortune. Changing the prompt text, the parser, or the layout all require editing the same function. Globals (`n`, `m`, `num`) couple the functions together so none of them can be understood alone.

```
def get_name():
    global n
    print(" WELCOME TO THE FORTUNE BOOTH ") # display
    x = input("What is your name? ")      # input
    n = x                                # shared state
    return x

def do_everything():
    get_name()
    love_fortune() # input + parse + pick + print
    job_fortune()  # input + parse + pick + print
    luck_fortune() # input + parse + pick + print
```

Listing 3. Mixed jobs and globals instead of a clear pipeline.

How the fixed file follows SRP

The fixed program is a pipeline. Input helpers only read and validate. `build_reading` only chooses lines. `print_reading` only displays. `main` is the only function that sequences the others. There are no globals. Each function has one reason to change.

Function	Single job
<code>prompt_nonempty / prompt_month / prompt_lucky_number</code>	Ask for one kind of input until it is valid.
<code>parse_month</code>	Turn text into a month number. No printing.
<code>choose_line</code>	Pick one string from a pool using a seed.
<code>collect_visitor</code>	Gather the three answers. Does not pick fortunes.
<code>build_reading</code>	Build the three fortunes. Does not print or prompt.
<code>print_reading</code>	Display the reading. Does not collect or choose.
<code>main</code>	Connect the steps in order.

```
def main() -> None:
    name, month, lucky_number = collect_visitor()
    reading = build_reading(month, lucky_number)
    print_reading(name, reading)
```

Listing 4. `main()` only orchestrates. Each callee does one job.

5. Document your code (and common commenting errors)

How the issues file documents badly

The issues file is full of comments, which is not the same thing as being documented. Lecture-style commenting errors are planted on purpose:

- **Noise / narration.** Comments such as `# this function gets the name`, `# strip spaces`, and `# ask for name` repeat the next line of Python. They add reading time and go stale the moment the code changes.
- **Wrong or stale comments.** The file header is dated 2019. The function `get_name` is labeled as returning the visitor's age, but it returns a name. A reader who trusts the comment is misled.

- **Commented-out code.** `# i = i + 1, # print("debug name=", x), # save_to_cloud(n, m, num)` should be deleted. Git already keeps history.
- **Wall-of-text headers.** The file opens with a changelog that describes printing and a database that does not exist. None of it helps someone change the fortunes.
- **Silence where a comment was actually needed.** `(m + num) % 4` is the non-obvious part, and it has no explanation.

```
def get_name():
    # this function gets the name
    x = input("What is your name? ")
    # strip spaces
    x = x.strip()
    # add 1 to nothing (leftover debug)
    # i = i + 1
    # print("debug name=", x)
    n = x
    # returns the visitor's age
    return x
```

Listing 5. Narrating comments, dead code, and a comment that lies about the return value.

How the fixed file documents well

The fixed file uses a module docstring for the program's purpose, and a one-line docstring on each function for the contract (what it does, what it returns). There is no comment on `text.strip()`. The one block comment that is more than a sentence explains *why* the seed is built from month and lucky number: so the same visitor gets the same reading twice. That is intent, not syntax.

```
def choose_line(lines: tuple[str, ...], seed: int) -> str:
    """Pick one line from a pool using a stable seed.

    The seed is built from the visitor's month and lucky number so the same
    person gets the same reading if they run the program twice.
    """
    return lines[seed % len(lines)]
```

Listing 6. A docstring that records a design choice, not the obvious return.

Commenting error	Issues file	Fixed file
Narrating the next line	<code># strip spaces</code> then <code>.strip()</code>	No comment. The method name is enough.
Stale / wrong comment	Returns age; header dated 2019	Docstrings match the actual return types.
Commented-out code	Debug prints, <code>save_to_cloud</code>	None. Unused ideas were deleted.
Missing why	Magic <code>% 4</code> with no note	Seed stability explained on <code>choose_line</code> .

6. Side-by-side summary

Topic	fortune_teller_issues.py	fortune_teller_fixed.py
DRY	Three cloned fortune functions; month parser pasted three times.	Shared <code>parse_month</code> and <code>choose_line</code> ; fortunes are data.
SRP	Each fortune function prompts, parses, picks, and prints. Globals glue them together.	Collect → build → print. <code>main</code> only sequences those steps.
Documentation	Narrating comments, a lying comment, commented-out code, stale changelog.	Module + function docstrings. One why-comment on seeding.
Visitor experience	Asks month and lucky number once per category (three times).	Asks each question once, then prints all three readings.

The two files are meant to be read together. The issues file is not 'bad because it is messy looking.' It is bad because a small change — a new month alias, a fifth fortune, a different prompt — has to be made in several places, by a reader who cannot trust the comments.

7. Bonus: GitHub collaboration evidence

The project lives in a public repository under **dsamanta12344**. Work was split onto feature branches, reviewed as pull requests, and merged into `main`.

- Repository: <https://github.com/dsamanta12344/coding-best-practices-activity>
- Pull request #1 (merged): <https://github.com/dsamanta12344/coding-best-practices-activity/pull/1> — issues version from `feature/violations`
- Pull request #2 (merged): <https://github.com/dsamanta12344/coding-best-practices-activity/pull/2> — fixed version from `feature/fixes`

Commit history on `main` (oldest first): README, violating fortune teller, cleaned-up fortune teller, two merge commits from the pull requests. Screenshots below were taken from the public GitHub UI.

```
*   cafccaf Merge pull request #2 (feature/fixes -> main)
| \
| * 4c6cda7 Add fortune teller that follows DRY, SRP, and documentation
* | b63bf78 Merge pull request #1 (feature/violations -> main)
| /
| * 6599287 Add fortune teller that violates DRY, SRP, and documentation
| /
* 05fd678 Add assignment README and gitignore
```

Listing 7. Local git graph matching the GitHub commit history.

The screenshot shows the GitHub repository home page for `dsamanta12344 / coding-best-practices-activity`. The repository is public and has 3 branches and 0 tags. The main branch is selected. The repository contains the following files:

File	Role
<code>fortune_teller_issues.py</code>	Same program, written to violate the three principles

The README file is titled "Coding Best Practices Activity" and describes a pair of command-line fortune tellers used to demonstrate three programming principles:

1. DRY (Don't Repeat Yourself)
2. Single Responsibility
3. Document your code (including common commenting errors)

The right sidebar shows the repository's statistics: 0 stars, 0 forks, and 0 watchers. The languages section shows Python at 100%.

Figure 1. Repository home: both Python files on main, public under dsamanta12344.

The screenshot shows the GitHub repository page for `dsamanta12344 / coding-best-practices-activity`, specifically the Branches section. The page displays the following branches:

Branch	Updated	Check status	Behind / Ahead	Pull request
main	now		Default	
feature/fixes	1 minute ago		3 0	#2
feature/violations	1 minute ago		3 0	#1

Figure 2. Branches: main (default), feature/fixes, and feature/violations, each linked to a pull request.

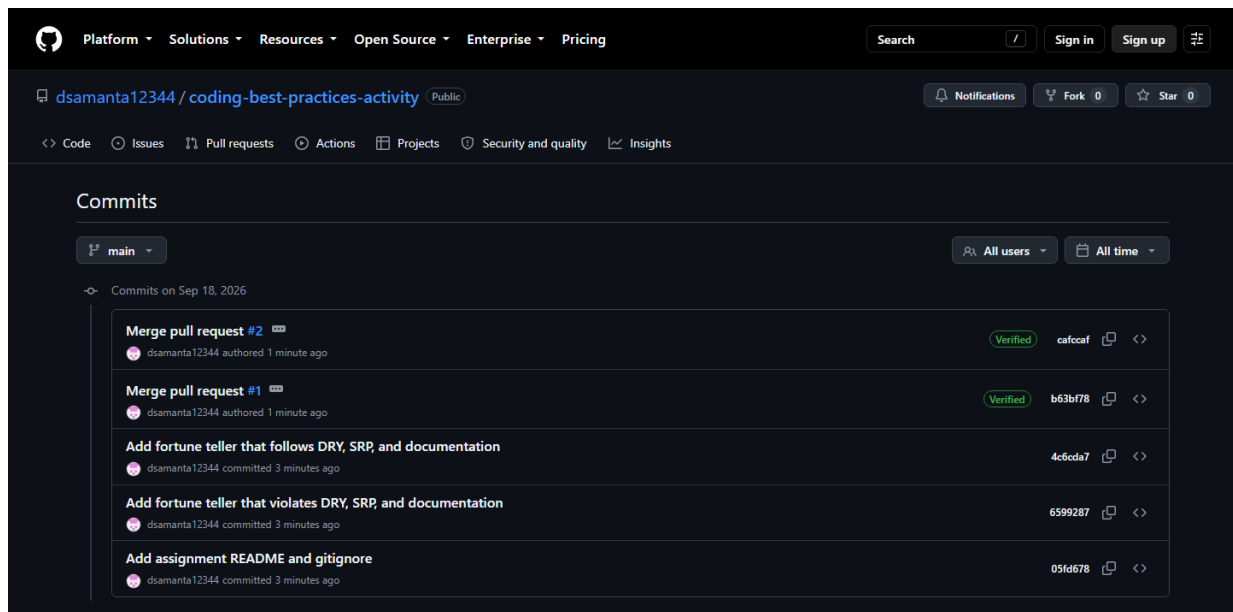


Figure 3. Commit history on main, including both feature commits and the two merge commits.

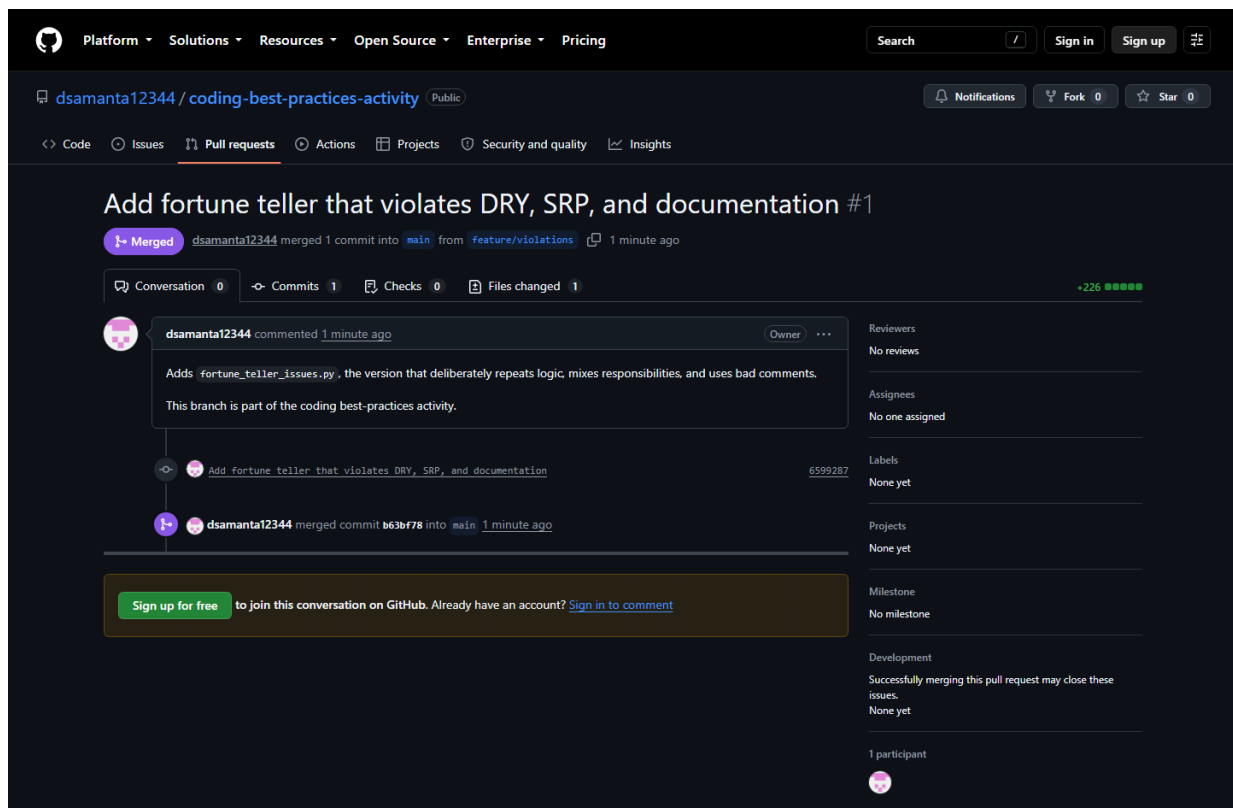


Figure 4. Pull request #1 merged: fortune_teller_issues.py from feature/violations.

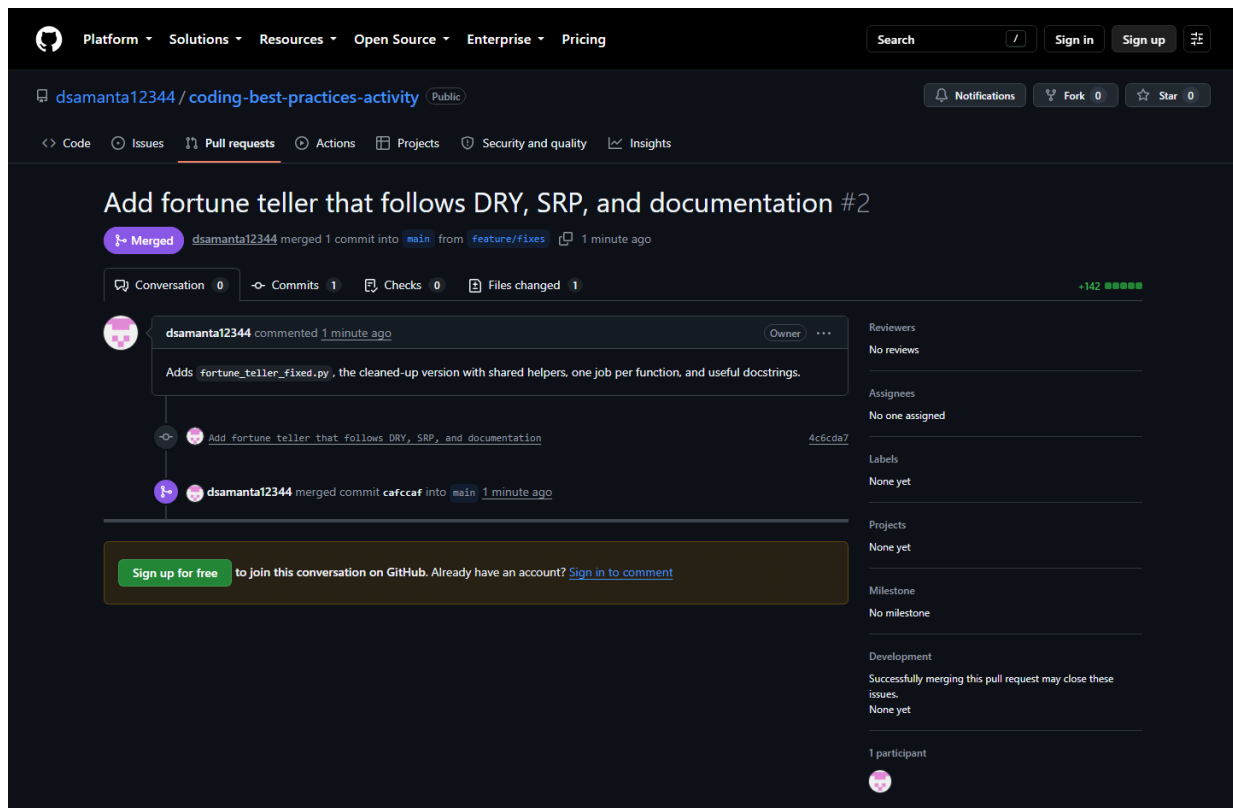


Figure 5. Pull request #2 merged: `fortune_teller_fixed.py` from `feature/fixes`.

8. What to submit

- This PDF: `Coding_Best_Practices_Writeup.pdf`
- Issues program: `fortune_teller_issues.py`
- Fixed program: `fortune_teller_fixed.py`
- GitHub link (includes the files, branches, commits, pull requests, and screenshots):
<https://github.com/dsamanta12344/coding-best-practices-activity>